Finale file formats

A reader's specification for `.mus` and `.musx`

Reconstructed by the finale-file-parser project. All hex dumps are synthetic.

Contents

1. Scope and provenance
2. Which formats are supported
3. Byte order, compression, and encryption
4. The `.mus` file header
5. Container: how the payload is found
6. Records: framing and addressing
7. The entry pool
8. The `.musx` container
9. Record catalog

1. Scope and provenance

This document describes the binary layout of Finale's `.mus` and `.musx` files as reconstructed by the `finale-file-parser` project. Finale was discontinued in 2024 and its format was never published; everything here comes from analyzing a curated corpus, from two Coda documents[1][2] vendored into the repository, and from prior community research.[3]

It is written for someone implementing a reader. Every structure is given as a C-style declaration, a field table, and a hex dump.

> ***Confidence.*** *Each structure below is annotated with what it was verified against. Where this project's findings contradict Coda's own documentation, that is stated explicitly rather than quietly resolved — there is one such case, the note alteration encoding in §7.2.*

No corpus bytes appear in this document

Every hex dump is **synthetic**, constructed by the generator that produced this PDF. The test corpus is licensed third-party music and its bytes are not reproducible here. Synthetic data also lets each dump isolate the field under discussion instead of burying it in a real record.

> **This document is generated from the parser.** Offsets, sizes and constants are imported from the reading code rather than retyped, so a layout here cannot silently drift from the implementation that reads it.

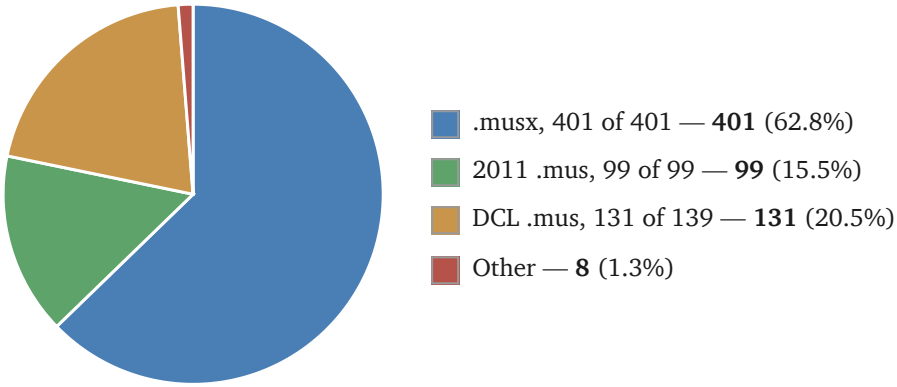
2. Which formats are supported

Three containers, spanning roughly two decades of Finale releases.

CONTAINER	FINALE ERA	PAYLOAD CODEC	BYTE ORDER	RECORD ENCODING
`.mus` (DCL)	2001–2005	PKWARE DCL (“implode”)	writing platform's	fixed 16-byte ETF rows
`.mus` (2011)	2011–2012	zlib (chained streams)	little-endian	self-identifying variable-length
`.musx`	2012–2024	ZIP + XOR cipher, then zlib	little-endian	EnigmaXML elements

Corpus coverage

The figures the parser is measured against, over 639 documents. 631 build a score.



The other eight files

Six empty scores

These files appear to be empty. They carry staves and measures, but no notes. The reader refuses them rather than returning an empty score, because an empty result is indistinguishable from a parse that went wrong.

One mirror

Mirror is Coda's own term — their 1996 documentation[4] warns that “mirrors and voice 2 create complications”, and Finale shipped a Mirror Tool for creating them.

A mirror is a staff that *displays another staff's music instead of holding its own copy*. An engraver reaches for one when two parts play the same thing — a doubled line, a cue, a piano reduction of what the winds are doing. Rather than duplicating the notes, the second staff is pointed at the first, so editing the original changes both.

Stored, this means exactly what it sounds like: there is one set of entries, and two `gfhold` records name the same entry span. Nothing marks either as the copy. To a reader walking the frame chain, the same entries simply turn up twice, in two different places in the score.

Five DCL documents in the corpus contain mirrors; four read fine, because their mirrored spans are named once. The one that fails is the only document where *two* `gfhold` records name the same entry span, which would require a single entry to exist in two places at once. The intermediate representation gives an entry exactly one location, so supporting this is a design change rather than a decoding problem.

One with a measure count mismatch

A file has 36 measures declared in the `measSpec` records, but measures as high as 111 appear in the `gfhold` records. The measures being referenced are not in the file, so most of the score cannot be reconstructed.

> **Not covered.** Finale versions between 2005 and 2011, and before 2001, are absent from the corpus and therefore absent from this document. Nothing here should be assumed to hold for them.

3. Byte order, compression, and encryption

Byte order is determined by the underlying architecture

For DCL-era `.mus` files, integers are written in the byte order of the machine that saved the file: **big-endian on Mac, little-endian on Windows**. This governs the pool records and every field inside them.

It is *detected, not assumed*, and the first pool record makes that possible. Its `kind` field is always 15, and 15 in a 16-bit field is `0f 00` in little-endian and `00 0f` in big-endian. So a reader tries one order, and if the first two bytes do not give 15 it tries the other: the wrong order yields 3,840, which is not a pool kind. One field, read two ways, decides the whole file.

The check that the choice was right is that the **chain walks to the last byte exactly**. Each pool record's `length` says how far the next one begins, so a reader adds lengths from `0x200` and lands on record after record. If the byte order were wrong the lengths would be wrong, and the walk would overshoot the end of the file or stop short of it. Landing precisely on the final byte, with no gap and no overrun, is strong evidence that every length was read correctly. All 139 DCL documents in the corpus do this: 102 little-endian, 37 big-endian.

> All files in the 2011 era are **little-endian**, since this is after Apple switched to Intel-based Macs.

Compression

DCL era (2001–2005)
: A chain of PKWARE DCL (“implode”) records beginning at `0x200`. No Python standard-library module reads this format. This project **wrote its own decoder**, an independent Python port of Mark Adler's `blast.c` from zlib's `contrib/blast`, whose correctness is pinned by that implementation's own published test vector.

2011 era
: A chain of raw zlib streams, the first located by scanning for the `78 9c` header. The standard library reads these.

`.musx`
: A ZIP archive. The member `score.dat` is XOR-obfuscated, and decodes to zlib-compressed EnigmaXML.

A DCL payload decompresses to about 3.3–4.5 times its stored size, and a 2011 payload to about 5.9–8.6 times. Decoded payloads in the corpus run from 32 KB to 683 KB.

Reading a file

Every input is hostile until parsed. A specification that describes only well-formed files leaves a reader open to three distinct attacks, which need three distinct defenses.

Decompression bombs

A small file can declare an enormous decompressed size, exhausting memory before anything is validated. Neither DCL nor zlib bounds its own output. This parser **rejects any score whose decoded payload exceeds 64 MiB**, counted across the whole pool chain rather than per stream, so a chain of individually modest pools cannot add up to an unbounded total. The largest real payload in the corpus is 683 KB, so the limit sits about 90 times above anything legitimate.

XML entity attacks

A `.musx` carries XML, and the XML standard's entity mechanism leaves parsers susceptible to two well-known attacks. A *billion laughs* attack[5] defines nested entities that expand exponentially, so a few kilobytes of markup inflate to gigabytes in memory. An *XML external entity* (XXE) attack[6] declares an entity pointing at a local file or a network address, and a parser that resolves it reads that file or makes that request on the attacker's behalf. Neither was intended by the standard; both follow from a feature it does include. All XML in this project is parsed with **`defusedxml`**, which disables entity expansion and external references. This is the project's only runtime dependency, and it exists for this reason alone.

Malformed offsets and lengths

Every offset and length in this document is read *from the file*, and may be incorrect. A record can declare a length running past the end of its pool, a frame can name entries that do not exist, and a walk can be steered into an infinite loop. Each such value is bounds-checked before use, and a file that fails a check raises a clear error naming what was wrong — never a crash, a hang, or a silent truncation.

The `score.dat` obfuscation

`score.dat` is XOR-ed with a keystream from a BSD `rand()` linear congruential generator, whose state advances as `state = state × 0x41c64e6d + 0x3039` (mod 2^{32}), starting from the fixed seed `0x28006d45`. The generator is restarted from that same seed at every 128 KiB boundary, so the keystream is one 128 KiB block repeated end to end for the whole file.

> *These cipher parameters were not discovered by this project. They come from denigma (MIT), whose source credits Deguerre.*

4. The `.mus` file header

Both `.mus` eras share a header. Its first 160 (`0xA0`) bytes carry the version banner and two provenance stamps.

A synthetic .mus header. The banner is the primary version evidence; the two provenance stamps say which application and platform wrote the file.

```
struct MusFileHeader {
    char[64]  banner;                   // +0x20   NUL-terminated; tail of a previ...
    uint32    created.date;             // +0x66   creation date stamp
    char[4]   created.app;              // +0x70   application tag, e.g. FIN
    char[4]   created.platform;         // +0x74   MAC or WIN
    uint32    modified.date;            // +0x8c   last-modified date stamp
    char[4]   modified.app;             // +0x96   application tag
    char[4]   modified.platform;        // +0x9a   MAC or WIN
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	`0x20–0x5f`	64	`char[64]`	`banner`	NUL-terminated; tail of a previous longer banner may follow
	`0x66–0x69`	4	`uint32`	`created.date`	creation date stamp
	`0x70–0x73`	4	`char[4]`	`created.app`	application tag, e.g. FIN
	`0x74–0x77`	4	`char[4]`	`created.platform`	MAC or WIN
	`0x8c–0x8f`	4	`uint32`	`modified.date`	last-modified date stamp
	`0x96–0x99`	4	`char[4]`	`modified.app`	application tag
	`0x9a–0x9d`	4	`char[4]`	`modified.platform`	MAC or WIN

Example bytes (160 shown), tinted to match the table above.

```
0000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0020  46 69 6e 61 6c 65 28 52 29 20 32 30 30 35 20 66  Finale(R) 2005 f
0030  6f 72 20 57 69 6e 64 6f 77 73 00 00 00 00 00 00  or Windows......
0040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0060  00 00 00 00 00 00 80 3a 09 00 00 00 00 00 00 00  .......:........
0070  46 49 4e 00 57 49 4e 00 00 00 00 00 00 00 00 00  FIN.WIN.........
0080  00 00 00 00 00 00 00 00 00 00 00 00 10 3b 09 00  .............;..
0090  00 00 00 00 00 00 46 49 4e 00 57 49 4e 00 00 00  ......FIN.WIN...
```

> The banner field is fixed-size and is *not* zero-filled when Finale rewrites it, so a shorter banner can leave the tail of a previous, longer one behind. Everything from the first NUL onward must be discarded.

The banner is the primary version evidence. A regular expression of the form `Finale\(R\)\s+(\d{4})` against the banner text yields the year. An unrecognized banner should leave the year unset with the raw text preserved, rather than failing — an unknown variant stays inspectable that way.

5. Container: how the payload is found

A `.mus` payload is a handful of compressed *pools*, laid end to end.

POOL	KIND	HOLDS
others	15	most record types, keyed by one cmper (a record's key; see §6)
details	16	records keyed by a pair of cmpers
entries	17	the notes themselves
text	18	human-readable strings

DCL era: a labeled chain

Records run from `0x200` to the last byte of the file, with no gaps. The DCL era **labels** its pools: every pool record opens with a two-byte `kind` field naming which pool follows — 15 others, 16 details, 17 entries, 18 text.

A DCL-era pool record. Records lie end to end from 0x200 to the last byte of the file, with no gaps.

```
struct DclPoolRecord {
    uint16   kind;                      // +0x0    15 others, 16 details, 17 entri...
    uint32   length;                    // +0x2    whole record, this 10-byte head...
    uint32   checksum;                  // +0x6    omitted entirely when the pool...
    uint8[]  stream;                    // +0xa    PKWARE DCL data, length - 10 bytes
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	kind	15 others, 16 details, 17 entries, 18 text
	0x02–0x05	4	uint32	length	whole record, this 10-byte header included
	0x06–0x09	4	uint32	checksum	omitted entirely when the pool is empty (length == 6)
	0x0a–0x10	7	uint8[]	stream	PKWARE DCL data, length - 10 bytes

Example bytes (17 shown), tinted to match the table above.

```
0000  0f 00 11 00 00 00 9c 21 44 00 00 04 0e c5 b3 9a  .......!D.......
0010  2f                                               /
```

> **length counts its own header.** The chain is walked by adding length to the current position. This is also why a fixed 0x20A works as "where the first stream starts": 0x200 plus the ten-byte header.

> A length of exactly 6 means the pool is **empty** — kind and length and nothing else, no checksum and no stream.

2011 era: an unlabeled chain

The first zlib stream is found by scanning for the `78 9c` header, and the rest follow one after another. There is no `kind` field, so nothing in the container says which pool a given stream holds.

A reader therefore identifies each pool **by recognizing its shape** rather than by counting positions. Each stream is tried against the structure a pool is expected to have, and the one that parses is that pool: a stream is the `others` pool if it walks cleanly as a run of self-identifying records and yields a plausible number of them, and the `entries` pool if it divides exactly into 38-byte slots with consistent entry numbers. A stream that satisfies neither is left alone.

The order *is* consistent in practice. Across all 99 2011-era corpus documents the `others` pool is the first stream, without exception, and the text pool is last in 98 of them. So a reader keyed to position would work on every file measured here.

> Recognition is still the right choice, because position is an assumption a file can violate and shape is not. A reader relying on order would fail *silently* on the first file that broke the pattern — producing a wrong score rather than an error — and 99 documents from a two-year window is thin evidence for a rule the format never states. More data would raise confidence in the ordering; it would not make the ordering a guarantee.

6. Records: framing and addressing

What a record is

A record is an object that contains three things:

A tag
: What kind of record this is — `measSpec`, `gfhold`. The full vocabulary is catalogued at the end of this section.

One or more keys
: What a record is about. A `measSpec` keyed 7 is measure 7; a `gfhold` keyed (3, 7) is staff 3, measure 7.

A payload
: Bytes whose meaning depends entirely on the tag. There is no self-description inside a payload: without the tag, the bytes mean nothing.

Three consequences matter to anyone writing a reader.

There are no pointers. Nothing holds a file offset. A `gfhold` does not point at a `frameSpec`; it contains the number 41, and somewhere there is a `frameSpec` whose key is 41. Resolution is a lookup by key equality — a join, not a dereference. Even the entry pool's `next` and `prev`, which look like a linked list, are entry *numbers*. The physical order of every record in a file could be shuffled without losing anything.

Records do not nest. Composition happens by reference. The music of a measure is not inside the measure's record; it is reached by a chain of key lookups.

A logical record is not always a physical one. In the 2011 era they are one to one — each record carries its own length. In the DCL era a record is a fixed 16-byte row, and anything larger continues into further rows under the same tag and key. ETF calls each row an *incidence*. So a record is the concatenation of its rows, and fields are addressed by offset into that concatenation, not into any one row.

The hierarchy, and where containment actually happens

"A pool contains records, and records contain entries" is half right. Pools do contain records. But an entry **is** a record: it lives in its own pool, keyed by entry number, a peer of `measSpec` rather than a child of anything.

`frameSpec` is what makes this look like containment. Reading "a frame holds entries 101 to 108", it is natural to picture the entries sitting inside the frame. They do not. The `frameSpec` record's payload is eight bytes: the number 101 and the number 108. The entries are in a different pool entirely, each keyed by its own number, and the `frameSpec` merely names two of those keys.

Concretely: a `frameSpec` is the same size on disk whether it names two entries or two hundred. Delete one and entries 101 to 108 are still in the entry pool, unchanged and readable — there is simply nothing left saying which staff and measure they belong to.

```
file
└── pools                    others · details · entries · text
    └── records              a pool is a flat run of them
        └── (entries pool)
            └── entry        a record: one chord or rest at one point in time
                └── notes    6-byte note records — real containment
```

Notes are the one genuine nesting in the format. An entry's payload holds its own `noteCount` and the note records themselves; they have no independent key and cannot be addressed from outside.

Everything else a score holds lives in a record's *payload*, as fields. A lyric syllable, a staff name and a page margin are all payload bytes of some record, addressed by offset once the tag is known. What makes an entry's notes different is that they are a *repeated substructure*, counted by a field of their parent, rather than fields at fixed offsets.

> In one line: **pools contain records, records reference each other by key, and a record's payload holds its fields — of which only an entry's notes are themselves a repeated substructure.**

2011 era: self-identifying records

A measSpec (tag 176) describing measure 1, belonging to the score rather than to a linked part. One record occupies 10 + length + 4 bytes.

```
struct MusRecord2011 {
    uint16   tag;                       // +0x0    record type; .musx names the sa...
    uint16   cmper;                     // +0x2    the (n) in an ETF ^XX(n) — the...
    uint16   part;                      // +0x4    0 for the score, then 1, 2, ......
    uint32   length;                    // +0x6    payload size in bytes
    uint8[]  payload;                   // +0xa    length bytes
    uint32   trailer;                   // +0x16   the length again
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
0x00–0x01	2	uint16	tag	record type; .musx names the same types
0x02–0x03	2	uint16	cmper	the (n) in an ETF ^XX(n) — the record's key
0x04–0x05	2	uint16	part	0 for the score, then 1, 2, ... per linked part
0x06–0x09	4	uint32	length	payload size in bytes
0x0a–0x15	12	uint8[]	payload	length bytes
0x16–0x19	4	uint32	trailer	the length again

Example bytes (26 shown), tinted to match the table above.

```
0000  b0 00 01 00 00 00 0c 00 00 00 60 00 58 02 04 00  ..........`.X...
0010  01 00 00 00 00 00 0c 00 00 00                    ..........
```

> These records are **self-identifying**: each carries its own key, so nothing outside the record is needed to address it — no directory, no key array, no positional convention.

> Records of one tag sit together in a section, and sections are separated by runs of two-byte zeroes, which a walk must skip. A zero there cannot begin a record: it would mean tag 0, which no record uses. Measured across the corpus, 6,372 of 6,416 such runs are followed by a *different* tag, so they mark section boundaries rather than occurring at random. Most are a single two-byte unit, and skipping one lands the walk on a four-byte boundary 85% of the time, which points at alignment before a new section — though the 15% that do not leave that unproven. This is the 2011 era only: the row-based DCL era pads differently, filling out a record's last row.

DCL era: fixed ETF rows

The same information, encoded completely differently. Where a 2011 record carries its own length, a DCL row is always 16 bytes and a long record simply continues into the next row.

A 2001–2005 others row: fixed 16 bytes, ETF's two-character tag.

```
struct DclOthersRow {
    uint16     cmper;                   // +0x0    the (n) in an ETF ^XX(n) — the...
    uint16     tag;                     // +0x2    two characters stored as a u16
    uint8[12]  data;                    // +0x4    six 16-bit values, or three 32-...
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	cmper	the (n) in an ETF ^XX(n) — the record's key
	0x02–0x03	2	uint16	tag	two characters stored as a u16
	0x04–0x0f	12	uint8[12]	data	six 16-bit values, or three 32-bit (ETF: twobytes and fourbytes)

Example bytes (16 shown), tinted to match the table above.

```
0000  01 00 53 4d 60 00 58 02 04 00 01 00 00 00 00 00   ..SM`.X.........
```

A record too big for one row **runs on into further rows** under the same tag and key. ETF calls each row an *incidence*, and a record's payload is its rows' data concatenated in file order. A reader knows where a record ends because rows are grouped and sorted by tag and key: the run stops at the first row whose tag or key differs. The expected number of rows is a property of each structure rather than anything the file states. Coda's specification states it for some record types but not all — a staff spec over three incidences, a page spec over two, a text block over four — and says nothing about the rest. The final row is zero-padded to fill it. Since the file never records that number, concatenating until the key changes is the reliable rule.

Worked example. A staff spec's transposition sits at offset 20 of the record. No row is 20 bytes long — each carries only 12 bytes of data — so offset 20 exists only once the rows are joined:

```
row 0  [cmper][tag][ 12 bytes ] → payload offsets  0..11
row 1  [cmper][tag][ 12 bytes ] → payload offsets 12..23
row 2  [cmper][tag][ 12 bytes ] → payload offsets 24..35
```

Offset 20 falls in **row 1, eight bytes into that row's data**. A reader does not compute that. It concatenates the rows' data into one 36-byte payload and reads offset 20 of it, exactly as it would for a 2011 record that arrived in one piece. Each row's own `cmper` and tag are addressing, not content, and are dropped before the join.

A details row. Details are addressed by a pair of keys rather than one. What the pair means depends on the tag: a frame hold is keyed by (staff, measure), a group spec by (instrument list, group id), a learned chord by (root, alternate bass), and an entry detail by the two halves of a 32-bit entry number.

```
struct DclDetailsRow {
    uint16     cmper1;                  // +0x0    first key
    uint16     cmper2;                  // +0x2    second key
    uint16     tag;                     // +0x4    two characters as a u16
    uint8[10]  data;                    // +0x6    five 16-bit values (ETF calls t...
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
0x00–0x01	2	uint16	cmper1	first key
0x02–0x03	2	uint16	cmper2	second key
0x04–0x05	2	uint16	tag	two characters as a u16
0x06–0x0f	10	uint8[10]	data	five 16-bit values (ETF calls them twobytes)

Example bytes (16 shown), tinted to match the table above.

```
0000  01 00 03 00 46 47 01 00 00 00 00 00 00 00 00 00  ....FG..........
```

How items point to each other

Addressing is by **key**, and the keys are small integers.

`cmper` ("comparator")
: A record's key — the `(n)` in ETF's `^XX(n)` notation. What it counts depends on the tag: for a `measSpec` it is the measure number, for a `staffSpec` the staff. The tag tables below give the key's meaning for every documented record.

`cmper1`, `cmper2`
: Details records are keyed by a *pair*, which is how a record hangs off an intersection — a `gfhold`, for instance, at (staff, measure).

`inci` ("incidence")
: When several records share a key, the incidence distinguishes them. In the DCL era it is also how one logical record spills across several 16-byte rows.

`part`
: 0 for the score; 1, 2, … for each linked part. A part's record overrides the score's for that part only.

Finding the music: the frame chain

This is the central traversal, and it is worth stating as a sequence:

1. A `gfhold` ("frame hold") is keyed by (staff, measure) and names up to four **frames**, one per layer.
2. Each frame id keys a `frameSpec`, which gives a `startEntry` and an `endEntry`.
3. Those are entry numbers into the entry pool. The entries between them, inclusive, are that layer's music for that measure.
4. Each entry carries its own `next` pointer, so the run can also be walked as a linked list.

```
for staff in staves:
    for measure in measures:
        hold = details["GF", staff, measure]     # absent => staff rests here
        if hold is None:
            continue
        for layer in 0..3:
            frame_id = hold.frame[layer]         # 0 => layer is empty
            if frame_id == 0:
                continue
            frame = others["FR", frame_id]
            for entry in entries[frame.startEntry .. frame.endEntry]:
                emit(entry, staff, measure, layer)
```

Four key lookups and no arithmetic on file positions. The entry range is inclusive at both ends.

Two ways to reach the same notes

A layer's entries are also chained: every entry carries `prev` and `next`, each holding another entry's number, so the entries of a layer form a doubly linked list running across the whole staff. Coda's documentation describes the pool this way, as entries "streamed together in a doubly linked list that roughly corresponds to a voice on a staff". The frame gives the same run's two endpoints.

Either route reaches the same notes, and the reader above uses the frame because it is the one that answers the question being asked. Walking the chain tells you what follows a given entry; it does not tell you which measure or staff you are in, since the chain runs straight through barlines. The `gfhold` is what supplies that, so a reader wanting music *by position* starts from the frame. A reader wanting to follow a voice forward — to resolve a tie, say — follows the chain instead.

> **An entry reached by more than one frame.** Two frames naming the same entry span is how a *mirror* is stored (§2) — one staff displaying another's music. Nothing in the record marks it as such: the two `gfhold` records are ordinary, and the only sign is that their frames resolve to the same entries.
> The record decodes without difficulty; what is hard is representing it. An intermediate form that gives each entry one location cannot hold an entry that sounds in two places, so this implementation refuses such a document rather than place the notes wrongly. That is a limitation of the representation, not of the decoding, and one of the eight non-building corpus documents is a case of it.

Known record tags

One vocabulary in three spellings: ETF's two-character tags, the 2011 era's numbers, and EnigmaXML's symbolic names.

Tier A — decoded and payload-confirmed

Parsed by this project, with fields verified against paired `.musx` documents.

NAME	TAG	ETF	POOL	DESCRIPTION
`clefOptions`	109		others	Clef definition table, read as a strided array
`articDef`	121		others	An articulation's definition; charMain is the glyph
`frameSpec`	146	`^FR`	others	Entry range for one layer of one measure
`instUsed`	159		others	The staves the score lays out, in layout order
`measSpec`	176	`^MS`	others	Per measure: width, key, beats, divbeat, barline
`repeatBack`	203		others	Backward repeat barline, keyed by measure
`repeatEndingStart`	204		others	Ending bracket, keyed by measure
`repeatPassList`	206		others	Which passes an ending is taken on
`staffSpec`	231	`^IS`	others	Per staff; transposition at +0x14
`articAssign`	1009		details	The articulation on an entry; names an articDef
`gfhold`	1044	`^GF`	details	(staff, measure) to clef and up to four frames
`staffGroup`	1057		details	Brace or bracket over a run of staves

NAME	TAG	ETF	POOL	DESCRIPTION
`tupletDef`	1072		details	Keyed by entry, not by (staff, measure)
`lyrDataVerse`	1108		details	Verse lyrics
`entry`	—	`^eE`	entries	The notes; fixed 38-byte slots

Named ETF tags

The 2001–2005 era uses ETF's two-character tags, and Coda documented many of them. The source column says which document named each one.

> *In this table, **ETF spec** is Coda's Enigma Transportable File Specification (reference 1); **LilyPond** is the LilyPond project's ETF format notes (reference 8); and **Cahill** is Cahill's thesis on Enigma and CPNView (reference 9). No LilyPond source code was read or used.*

> **Tag names are case sensitive.** `^AC` is Tempo and `^ac` is performance data; `^CH` is a chord and `^hC` a learned one; `^FB` is the fretboard library while `^fb` is an unrelated record with a different partner.

Others

TAG	NAME	KEY (CMPER)	SOURCE
`^MS`	Measure Spec	measure number	ETF spec
`^IS`	Staff Spec	instrument (staff) number	ETF spec
`^IU`	Instrument Used	IUList id; one incidence per slot	ETF spec
`^FR`	Frame	frame id; the entry span of one layer	LilyPond
`^AC`	Tempo	measure	ETF spec
`^DY`	Score Expression	measure	ETF spec
`^DI`	Separate Placement	measure of the score expression	ETF spec
`^DT`	Text Expression	dynumber in staff/score expression	ETF spec
`^DO`	Shape Expression	dynumber in staff/score expression	ETF spec
`^PD`	Play Dump	value in text/shape expression	ETF spec
`^SD`	Shape	shapedef in a shape expression	ETF spec
`^SL`	Shape Instructions	instlist; 3 instructions per record	ETF spec
`^SB`	Shape Data	datalist; the data those instructions use	ETF spec
`^TX`	Text Block	text block id; layout of a block of text	ETF spec
`^pT`	Page Text Block	page	ETF spec
`^PS`	Page Spec	page number	ETF spec
`^SS`	Staff System Spec	system number	ETF spec
`^MN`	MeasNumberRegion	which measure-number region	ETF spec
`^IV`	Chord Suffix	which chord suffix	ETF spec
`^IK`	Chord Playback	matches the cmper of its IV	ETF spec

TAG	NAME	KEY (CMPER)	SOURCE
^IX	Articulation Definition	referenced by an IM	ETF spec
^Sx	Slur	slur number; four rows per slur	LilyPond
^FB	Fretboard library	1–192; a fixed table, see below	measured

DETAILS

TAG	NAME	KEYS	SOURCE
^GF	Frame Hold	(staff, measure)	LilyPond
^NG	Group Spec	(iuList, groupID)	ETF spec
^FL	Floats: independent key and time	(instrument, measure)	ETF spec
^mt	Measure Text Block	(instrument, measure)	ETF spec
^LP	Staff Enduction	(staff system, instrument)	ETF spec
^MI	MeasNumberSeparate	(instrument, measure)	ETF spec
^ME	Midi Expression	(instrument, measure)	ETF spec
^hC	Learned Chord	(root, alternate bass)	ETF spec
^Ex	Slur detail	two rows per slur, paired with Sx	Cahill
^GT	Fretboard index	root as MIDI 60–71; 16 incidences each	measured
^DF	Percussion map	(map id 1–26, MIDI note 0–127)	measured

ENTRY DETAILS

Keyed by an entry number rather than by position — the two cmpers hold the high and low words of the entry number.

TAG	NAME	CARRIES	SOURCE
^ac	Performance data	MIDI velocity and duration, per note	ETF spec
^ED	Staff Expression	one per expression on the entry	ETF spec
^CD	Cross Staffing	note moved to another staff	ETF spec
^IM	Articulation	positions a mark; names an IX	ETF spec
^CH	Chord	chord symbol on the entry	ETF spec
^CN	Notehead Mods	per-note custom attributes	ETF spec
^ve	Lyric: verse	syllable offset into a raw text record	ETF spec
^ch	Lyric: chorus	as ve	ETF spec
^se	Lyric: section	as ve	ETF spec

^FB and ^GT were identified by structure, not by a document. Neither appears in any source consulted. They were matched because FB holds **exactly 192 records in every document that has it**, keyed contiguously 1–192 — a fixed table rather than per-score data — and EnigmaXML's `fretboardSymbol` holds exactly 192 in every document too. An invariant of that kind survives the two corpora being different sizes and different repertoire, which ordinary frequency comparisons do not.
GT is its index: twelve records keyed 60–71, the chromatic pitch classes from middle C, with sixteen incidences each. Twelve roots times sixteen shapes is 192. FB and GT appear in exactly the same 44 of 139 documents and in no others, and nothing else shares that set.

The fretboard library is written only when the feature is used: 44 of 139 DCL documents carry it, against 149 of 150 `.musx` documents, later versions shipping the defaults regardless.

^DF is a percussion map. It is keyed by a map id (1–26) and a MIDI note (0–127), and gives that note's appearance on a percussion staff. It is the most common record in a DCL document, which a table of roughly twenty-five maps by 128 notes accounts for.
Three measurements support the reading. The id tops out at 21 or 25 whatever a document's staff count, so it selects from a fixed library rather than describing a staff. The payload's first field repeats its own key in 9,396 of 10,434 non-empty records. And 79% of non-empty entries fall in notes 35–81, the General Midi percussion range, which is only 37% of the key space. The first in any document sits at note 35, General Midi's first defined percussion note, with everything below it empty.

The remaining payload fields, presumably notehead and staff position, are not decoded here.

Not every DCL tag is two printable characters. Alongside the character tags, the details pool holds thirty **numeric** tags in three regular runs — `0x8001`–`0x800a`, `0x9001`–`0x900a` and `0xa001`–`0xa00a` — each appearing about once per document in all 139 DCL documents, and in both byte orders. Their meaning is not established here; what matters for an implementer is that the field is a u16, and only sometimes a pair of letters.

A variant described in a third-party document and not observed. The LilyPond format notes describe a second form of ^GF occupying a single row, laid out `frame, clef, ...` rather than `clef, flags, frames`. A reader assuming the two-row form would take a frame id for a clef.
Measured across all 139 DCL corpus documents: every one of the 14,191 GF records has exactly two incidences and a 20-byte payload. The single-row form occurs in none of them.

Two readings fit. The variant may be real and simply absent from this corpus, in which case it is a gap. Or the note may be mistaken: it is one person's reverse-engineering write-up rather than vendor documentation, and being written down does not make it correct. It is recorded so a reader meeting such a file knows the possibility exists, not as an established part of the format.

Tier B — numeric tags named by key-sequence matching only

These names come from a different method, and it is worth being precise about what it does and does not establish.

A `.musx` and a 2011 `.mus` of the same music contain the same records. Reading the `.musx`, whose records are named, gives the sequence of `(cmper, part)` keys belonging to each name. Reading the `.mus` gives the same sequences against numeric tags. Where a numeric tag's key sequence matches a name's exactly, across many documents, the two are almost certainly the same record type. The “Docs” column is how many paired documents agreed.

What limits it is that a key sequence is not unique. Many record types are keyed one per measure, or one per staff, and any two of those produce identical sequences in a document. The match then says only "this tag has the same shape as that name", and two tags of the same shape can be swapped without the evidence noticing. No conflicts were observed — no tag matched two names, and no name matched two tags — but that is what would be expected either way, since a swap between two same-shaped tags is invisible to a test that only compares shapes.

So eighty documents agreeing that tag 124 has `channelPlayData`'s key sequence is strong evidence that tag 124 is *some* record keyed the way `channelPlayData` is keyed. It is not evidence about the meaning of its bytes, which is what confirming a record type requires. That is why Tier A, whose payload fields were read and checked against a paired document, is a different kind of claim.

TAG	NAME	DOCS
124	`channelPlayData`	80
126	`chordSuffixPlay`	85
131	`drumLibName`	86
134	`durAllot`	91
136	`execShape`	85
140	`fretboardSymbol`	91
144	`fontName`	5
147	`lockMeas`	81
149	`fretInst`	90
163	`layerAtts`	85
165	`metaArtic`	80
168	`metaDynam`	81
169	`metaKeySig`	80
170	`metaRepeat`	81
171	`metaShape`	80
172	`metaStaffStyle`	80
173	`metaTimeSig`	81
235	`shapeExprDef`	10
242	`textExpressionEnclosure`	14
315	`volumeValue`	14

> `fontName` (5 documents) and `shapeExprDef` (10) rest on so few agreements that they are better read as guesses.

Tier C — observed but unidentified

189 distinct tags appear in the 2011 `others` pool alone, of which the tiers above account for roughly thirty. The rest are read, keyed and carried, but their meaning is unknown. The most frequent, by record count across the corpus:

TAG	ERA	POOL	RECORDS
213	2011	others	25,576
215	2011	others	25,576
140	2011	others	18,624
214	2011	others	12,681
125	2011	others	12,612
148	2011	others	11,887
126	2011	others	11,208
183	2011	others	10,114
192	2011	others	7,353
241	2011	others	7,353
217	2011	others	6,638
1043	2011	details	166,524
1064	2011	details	8,162
1063	2011	details	2,162
1066	2011	details	1,483
1034	2011	details	1,164
1060	2011	details	1,156
^fb	DCL	details	63,324
^DN	DCL	details	18,647
^BC	DCL	others	3,809
^DA	DCL	others	2,963
^fg	DCL	others	2,884
^OC	DCL	others	2,170
^ls	DCL	others	1,809

Two hints from the counts. Several pairs share an exact record count — 213/215, 192/241, and `sL`/`sb` — which across a whole corpus usually means a definition-and-assignment couple like `articDef`/`articAssign`. And details tag 1043, at 166,524 records, is roughly sixteen times more common than `gfhold`, which suggests something per entry or per note rather than per measure.

What this document covers. *The tags named here are the ones on the path from a file to its notes, which is where the work started; they are not the most common ones.*
Counting distinct tags observed across the corpus: ***34 of the 227*** *numeric tags the 2011 era uses, and* ***34 of the 265*** *ETF tags the 2001–2005 era uses. Together* ***68 of 492****, about one in seven. What the remaining tags hold is not known.*

7. The entry pool

The entry pool contains the actual musical notes — both pitch and duration. It is the one structure the two `.mus` eras share field for field, differing only in the byte order their integers are written in.

Two units, and one word, used throughout

EDU — Enigma Duration Units
Coda's time unit, in which **1024 is a quarter note**. A whole note is 4096, a dotted quarter 1536.[7]

EVPU — Enigma Virtual Page Units
Coda's distance unit, **288 to the inch**. Positions and widths elsewhere in this document are in EVPU.[7]

Entry
In Coda's terminology an entry is **a note, a chord, or a rest**. A chord is one entry with several notes, not several entries.

The entry pool is a flat array of fixed 38-byte slots. Every slot repeats the entry number it belongs to, so a reader never has to track position.

```
struct EntrySlotFirst {
    uint32   entnum;                    // +0x0    this entry's number, its key
    uint16   slot;                      // +0x4    0 for this entry's first slot,...
    uint32   prev;                      // +0x6    previous entry number, 0 if none
    uint32   next;                      // +0xa    next entry number, 0 if none
    uint16   dura;                      // +0xe    written duration in EDU; 1024 =...
    int16    pos;                       // +0x10   manual positioning in EVPU, signed
    uint32   flags;                     // +0x12   SETBIT: a real entry; NOTEBIT:...
    uint16   extflags;                  // +0x16   extended flags
    uint16   noteCount;                 // +0x18   notes in this entry, across all...
    NoteRec  note[0];                   // +0x1a   pitch word + flags; see below
    NoteRec  note[1];                   // +0x20   the first slot holds two
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
0x00–0x03	4	uint32	entnum	this entry's number, its key
0x04–0x05	2	uint16	slot	0 for this entry's first slot, then 1, 2, ...
0x06–0x09	4	uint32	prev	previous entry number, 0 if none
0x0a–0x0d	4	uint32	next	next entry number, 0 if none
0x0e–0x0f	2	uint16	dura	written duration in EDU; 1024 = quarter note
0x10–0x11	2	int16	pos	manual positioning in EVPU, signed
0x12–0x15	4	uint32	flags	SETBIT: a real entry; NOTEBIT: note, not rest
0x16–0x17	2	uint16	extflags	extended flags
0x18–0x19	2	uint16	noteCount	notes in this entry, across all its slots
0x1a–0x1f	6	NoteRec	note[0]	pitch word + flags; see below
0x20–0x25	6	NoteRec	note[1]	the first slot holds two

Example bytes (38 shown), tinted to match the table above.

```
0000  65 00 00 00 00 00 64 00 00 00 66 00 00 00 00 04   e.....d...f.....
0010  00 00 00 00 00 c0 00 00 02 00 c0 03 00 00 00 00   ................
0020  01 04 00 00 00 00                                 ......
```

> **An entry is the musical object; a slot is the storage cell.** Every slot is 38 bytes, and an entry uses as many as its notes require. They hold different numbers of notes because the first spends bytes on the entry itself:
>
> first slot — 6 bytes of slot header, 20 bytes of entry fields, then room for 2 notes
> later slots — 6 bytes of slot header, then room for 5 notes
>
> So a three-note chord occupies two slots: two notes in the first and one in the second, with the rest of the second slot unused. An entry may carry up to 12 notes.

> **The two entry flags a reader must respect.** `SETBIT` (0x80000000) is set on every legitimate entry, so a slot without it is not one. `NOTEBIT` (0x40000000) distinguishes a note from a rest: when it is clear the entry sounds nothing, and a rest normally carries no note records at all.

The note record

TCD — Tone Center Displacement

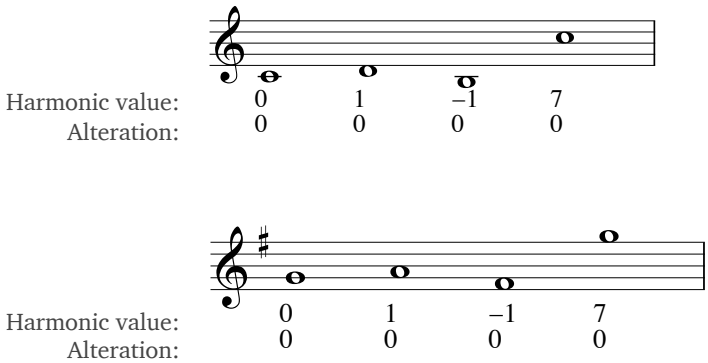
The first two bytes of a note record are a single 16-bit field Coda calls the **TCD**. It carries two things at once: which pitch, and how that pitch is altered against the key.

```
bit   15 14 13 12 11 10  9  8  7  6  5  4   3   2  1  0
     +-----------------------------------------+---+--------+
     |        harmonic value (12 bits)         | s | m m m  |
     +-----------------------------------------+---+--------+
        diatonic step, signed, 0 = tonic         ^   alteration
                                                 sign
```

The harmonic value

The harmonic value is a diatonic step relative to the current key. 0 is the tonic; −1 is a step below, while 7 is an octave up.

The value counts *steps of the key*, not semitones, so the same numbers name different pitches in different keys:

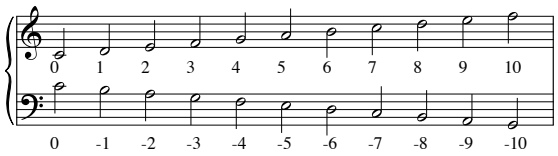


The same four harmonic values in G major (one sharp). The third is F sharp, which the key provides, so its alteration is 0 too.

This is why transposing a passage by changing its key signature moves every note with it and rewrites nothing: the stored numbers do not change.

How octaves are handled

There is no octave field. The harmonic value is a signed step count, so an octave is simply seven steps: 7 is an octave above the tonic, −7 an octave below, 14 two octaves above. Twelve bits give it a range of −2048…2047, which is far more than any notation needs.

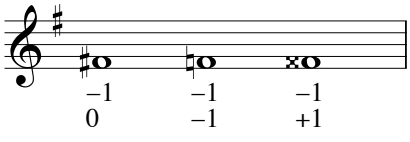


C major. Ascending 0 to 10 on the treble, descending 0 to −10 on the bass, both from the same note. Nothing marks the octave: past 7 the count simply continues.

The alteration

The alteration is how far the note departs from that diatonic step: 0 means the note the key gives, +1 a semitone above it, −1 a semitone below.

It is measured against the key, not against the printed accidental:



G major. One harmonic value, three alterations: F sharp as the key gives it, F natural a semitone below, F double sharp a semitone above.

So in G major **F♯ has an alteration of 0** — the key already provides it — and an F natural is **−1**, even though the natural is the sign the engraver prints.

> **The alteration is sign-and-magnitude, not two's complement.** Bit 3 is a sign, and bits 2–0 a magnitude from 0 to 7. Coda's documentation calls it “a signed quantity ... -8 to +7”, which reads as two's complement; the corpus disagrees, and the corpus wins.
> The two readings agree on naturals and sharps and diverge on every flat. `0x1` is +1 either way, but `0x9` = binary `1001` is **−1** — sign set, magnitude 1. Read as two's complement the same nibble is −7, which spells a note six steps away from the one Finale displays.

A note record: six bytes carrying a note's pitch and its properties.

```
struct NoteRec {
    uint16  tcd;                        // +0x0    Tone Center Displacement: pitch...
    uint32  flags;                      // +0x2    note properties; see the table...
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	tcd	Tone Center Displacement: pitch and alteration
	0x02–0x05	4	uint32	flags	note properties; see the table below

Example bytes (6 shown), tinted to match the table above.

```
0000  09 04 00 00 00 40                                  .....@
```

The note flags

The 32-bit field holds a note's properties. The id mask is the one an implementer meets soonest: entry details such as articulations and performance data address *one note of a chord* by that id.

BIT	CODA'S NAME	MEANING
0x80000000	SETBIT	legality; set on every real note
0x40000000	TSBIT	tie start
0x20000000	TEBIT	tie end
0x10000000	CROSSBIT	cross-staff: the note is drawn on another staff
0x08000000	UPSECBIT	upstem second
0x04000000	DWSECBIT	downstem second
0x02000000	UPSPBIT	on the upper stem, where stems are split
0x01000000	ACCIBIT	show an accidental; recomputed while editing unless frozen
0x00800000	PARENACCI	parenthesize that accidental
0x001F0000	TGFNID	mask: the note's id, 1–12, within its entry
0x00000002	FREEZEACCI	freeze the accidental bit in place

Durations

Durations are measured in **EDU**, where 1024 is a quarter note and 4096 a whole note. A dot adds half again, so a dotted quarter is 1536 and a double-dotted quarter 1792.

Every duration observed across the 238 corpus `.mus` documents — 108,466 of them — takes one of sixteen values:

EDU	NOTE VALUE	COUNT
8,192	breve	47
6,144	dotted whole	17
4,096	whole	1,043
3,584	double-dotted half	3
3,072	dotted half	977
2,048	half	5,266
1,792	double-dotted quarter	14
1,536	dotted quarter	2,318
1,024	quarter	34,960

EDU	NOTE VALUE	COUNT
768	dotted eighth	2,121
512	eighth	38,046
384	dotted 16th	89
256	16th	22,651
192	dotted 32nd	6
128	32nd	874
64	64th	34

8. The `.musx` container

A `.musx` is a ZIP archive. Unlike the two `.mus` eras it needs no bespoke container walking — a standard ZIP reader opens it — but its payload is encrypted.

MEMBER	CONTENTS
`mimetype`	the fixed string `application/vnd.makemusic.notation`
`META-INF/container.xml`	archive manifest
`NotationMetadata.xml`	document metadata
`score.dat`	**the score** — XOR-encrypted, then zlib
`graphics/*`, `presets/*`	embedded artwork and presets

Reading `score.dat`

1. Read the member's bytes from the archive.
2. XOR with the LCG keystream (§3), which resets every 128 KiB.
3. Inflate the result with zlib.
4. The output is **EnigmaXML**, an XML document whose elements carry the same record types the binary formats encode numerically.

> **Treat the archive as hostile.** A ZIP may declare member sizes that do not match its content, repeat member names, or expand enormously. Cap the inflated size, reject duplicate names, and never resolve a member path outside the archive root.

Why one reader covers all three

EnigmaXML names its record types symbolically — `measSpec`, `gfhold`, `entry` — where the 2011 era numbers them and the DCL era uses ETF's two-character tags. These are three spellings of one vocabulary. A reader that converges all three onto a single document model gets the rest of the pipeline for free, which is what this project does: the container differs, the music does not.

CONCEPT	.MUSX	2011 .MUS	DCL .MUS
measure	`measSpec`	176	`^MS`

CONCEPT	.MUSX	2011 .MUS	DCL .MUS
staff	`staffSpec`	231	`^IS`
frame	`frameSpec`	146	`^FR`
frame hold	`gfhold`	1044	`^GF`
note group	`entry`	entry pool	entry pool

9. Record catalog

Every record type this project decodes, with the offsets its reader actually uses. Fields not listed are present in the payload but not yet established; they are omitted rather than guessed at.

measSpec — tag 176, DCL `^MS`

measSpec — one per measure, keyed by measure number. 2011 tag 176; DCL tag `^MS`.

```
struct MeasSpec {
    uint16  width;                      // +0x0    measure width in EVPU
    uint16  key;                        // +0x2    key signature; 0 means inherit
    uint16  beats;                      // +0x4    time signature numerator
    uint16  divbeat;                    // +0x6    EDU per beat; 1024 = quarter
    uint16  flags;                      // +0xa    low byte holds the barline nibble
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	width	measure width in EVPU
	0x02–0x03	2	uint16	key	key signature; 0 means inherit
	0x04–0x05	2	uint16	beats	time signature numerator
	0x06–0x07	2	uint16	divbeat	EDU per beat; 1024 = quarter
	0x0a–0x0b	2	uint16	flags	low byte holds the barline nibble

Example bytes (12 shown), tinted to match the table above.

```
0000  58 02 03 00 04 00 00 04 00 00 02 00          X...........
```

> **A key of 0 is C major.** The field packs a mode in its high byte (0 major, 1 minor) and a signed count of sharps or flats in its low byte, so 0 is major with no accidentals. A `.musx` expresses the same thing by *omitting* its key element: across 401 documents and 19,644 key elements, not one is written as zero.

> **Absence is not inheritance.** A missing element reads naturally as “same as the measure before”, and it is not. One corpus document runs key=1 for measures 1–32, no element for 33–52, then key=1 again from 53. Opened in Finale, measure 33 begins a new song with a natural cancelling the sharp and chords of C, G7 and F. It is a key change to C major, not a continuation of G.

> **The corpus alone cannot settle this**, which is worth knowing before trying. The obvious test is to look for accidentals: music in C stored under an inherited G would need a flat on every F. But *no absent-key run anywhere in the corpus contains a single accidental* — every one is purely diatonic, so both readings fit the notes equally well. What decided it was rendering the page.

> **What a reader must not do** is carry the previous key forward across a measure that states none. That silently keeps a piece in the key it started in through every passage returning to C. This project made exactly that mistake: 18 passages, 741 measures and 11,421 entries were spelled a step sharp, and every test passed either way.

> The barline style is the **high nibble** of that byte, whose low bits carry the repeat flags. Observed values: **1** an ordinary barline and **2** a double bar, agreeing with the paired `.musx` on 3,960 ordinary measures and all 11 double bars. **3** would be the obvious reading for a final barline and does not occur once in 4,427 measures across 99 documents, so either no file here ends with one or it is stored elsewhere; the corpus cannot say which.

frameSpec — tag 146, DCL `^FR`

frameSpec — a run of entries forming one layer of one measure. 2011 tag 146; DCL tag ^FR.

```
struct FrameSpec {
    uint8[]  header;                    // +0x0    era-dependent lead-in
    uint32   startEntry;                // +0x6    first entry number in this frame
    uint32   endEntry;                  // +0xa    last entry number, inclusive
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x05	6	uint8[]	header	era-dependent lead-in
	0x06–0x09	4	uint32	startEntry	first entry number in this frame
	0x0a–0x0d	4	uint32	endEntry	last entry number, inclusive

Example bytes (14 shown), tinted to match the table above.

```
0000  00 00 00 00 00 00 65 00 00 00 6c 00 00 00        ......e...l...
```

> **The base offset differs by era: 4 for 2001, 6 for 2005.** The reader distinguishes them by incidence count — a three-incidence spec is the 2001 shape. Reading the wrong base yields entry numbers that look plausible and are wrong, which is the worst kind of error this format offers.

> When a spec has more than one incidence, a `startTime` u32 sits at +0, in EDU from the start of the measure.

gfhold — tag 1044, DCL `^GF`

gfhold ("frame hold") — keyed by two cmpers, (staff, measure). 2011 tag 1044; DCL tag ^GF. This is the record that connects a place in the score to its music.

```
struct GfHold {
    uint16  clefID;                     // +0x0    clef in force for this measure
    uint16  frame1;                     // +0x6    layer 1 frame id; 0 = layer empty
    uint16  frame2;                     // +0x8    layer 2 frame id
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	clefID	clef in force for this measure
	0x06–0x07	2	uint16	frame1	layer 1 frame id; 0 = layer empty
	0x08–0x09	2	uint16	frame2	layer 2 frame id

Example bytes (10 shown), tinted to match the table above.

```
0000  01 00 00 00 00 00 29 00 00 00                ......)...
```

> A frame of **0 means the layer is empty** and should be omitted, matching a .musx, where an absent slot reads as "this layer holds nothing".

> The format provides four layer slots. Only the first two are decoded here; no corpus document was found carrying a third or fourth.

staffSpec — tag 231, DCL `^IS`

staffSpec — one per staff, keyed by staff number. 2011 tag 231; DCL tag `^IS`. Only the transposition field is decoded.

```
struct StaffSpec {
    uint16  transposition;              // +0x14   written-to-sounding interval
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x14–0x15	2	uint16	transposition	written-to-sounding interval

Example bytes (22 shown), tinted to match the table above.

```
0000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0010  00 00 00 00 f9 ff                              ......
```

> **Transposition is the written-to-sounding interval**, as a diatonic step count: a B-flat clarinet sounds a step below what is written, a horn in F a fifth below. It is applied on top of the harmonic value a note already carries, which is why a transposing part and a concert-pitch part store the same numbers.

> **The octave is not in this field, and not anywhere else.** Searching every byte of every staffSpec across the paired documents found no byte that separates staves transposing by an octave from staves transposing by the same interval within one. Finale does not need it stored: the instrument itself fixes the octave, and a reader without an instrument table must supply that knowledge from outside the file.

> The staff's *name* is not here — it lives in the text pool, which is why a .mus converts with positional part names unless that pool is resolved.

tupletDef — tag 1072

tupletDef — 2011 tag 1072. The example reads “3 quarters in the time of 2 quarters”.

```
struct TupletDef {
    uint16  symbolicNum;                // +0x0    printed count, the 3 of a triplet
    uint16  symbolicDur;                // +0x2    printed unit in EDU
    uint16  refNum;                     // +0x4    count these replace
    uint16  refDur;                     // +0x6    unit they replace, in EDU
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
0x00–0x01	2	uint16	symbolicNum	printed count, the 3 of a triplet
0x02–0x03	2	uint16	symbolicDur	printed unit in EDU
0x04–0x05	2	uint16	refNum	count these replace
0x06–0x07	2	uint16	refDur	unit they replace, in EDU

Example bytes (8 shown), tinted to match the table above.

```
0000  03 00 00 04 02 00 00 04                        ........
```

> The sounded duration of an entry inside a tuplet is its written duration scaled by `(refNum × refDur) / (symbolicNum × symbolicDur)`.

staffGroup — tag 1057

staffGroup — the braces and brackets down the left edge. 2011 tag 1057.

```
struct StaffGroup {
    uint16  startInst;                  // +0x0    first staff in the group
    uint16  endInst;                    // +0x2    last staff, inclusive
    uint16  bracketId;                  // +0xa    bracket shape
    uint8   barlineFlags;               // +0x15   bit 0: barlines cross the group
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
0x00–0x01	2	uint16	startInst	first staff in the group
0x02–0x03	2	uint16	endInst	last staff, inclusive
0x0a–0x0b	2	uint16	bracketId	bracket shape
0x15	1	uint8	barlineFlags	bit 0: barlines cross the group

Example bytes (24 shown), tinted to match the table above.

```
0000  01 00 04 00 00 00 00 00 00 00 03 00 00 00 00 00  ................
0010  00 00 00 00 00 01 00 00                          ........
```

lyricVerse — tag 1108

lyricVerse — 2011 tag 1108. The payload is an array of 20-byte slots, one per entry that sings.

```
struct LyricVerseSlot {
    uint16  number;                     // +0x0    verse number
    uint16  syll;                       // +0x2    syllable index into the text pool
    uint16  wext;                       // +0x8    non-zero: word extends with a m...
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	number	verse number
	0x02–0x03	2	uint16	syll	syllable index into the text pool
	0x08–0x09	2	uint16	wext	non-zero: word extends with a melisma

Example bytes (20 shown), tinted to match the table above.

```
0000  01 00 07 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0010  00 00 00 00                                      ....
```

articAssign — tag 1009

articAssign — 2011 tag 1009. Points at an articDef (tag 121), which carries the glyph.

```
struct ArticAssign {
    uint16  definition;                 // +0x0    key of the articDef this uses
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	definition	key of the articDef this uses

Example bytes (4 shown), tinted to match the table above.

```
0000  0c 00 00 00                                      ....
```

instUsed — tag 159

instUsed — 2011 tag 159. A 24-byte slot per incidence, giving the score's staff order. This, not the staff number, is what a part list should follow.

```
struct InstUsedSlot {
    uint16  staff;                      // +0x0    staff number, one per incidence
    uint16  staff[1];                   // +0x18   the next slot, 24 bytes on
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	staff	staff number, one per incidence
	0x18–0x19	2	uint16	staff[1]	the next slot, 24 bytes on

Example bytes (48 shown), tinted to match the table above.

```
0000  01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0010  00 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00  ................
0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Recognized but only partly decoded

These records are read and their keys resolved, but their payload layouts are not fully established. They are listed so an implementer knows they exist rather than meeting them as unexplained bytes.

RECORD	TAG	WHAT IT CARRIES
`articDef`	121	the glyph an articAssign refers to; character offsets 0 and 2
`repeatBack`	203	backward repeat barline; target measure
`repeatEndingStart`	204	first and second ending brackets
`repeatPassList`	206	which passes an ending applies to
`clefOptions`	109	the clef definition table, read as a strided array

References

References

1. *Enigma Transportable File Specification*, Coda Music Technologies. Identified by the Library of Congress as version 98c.0, for Finale 97 for Mac and Windows: `loc.gov/preservation/digital/formats/fdd/fdd000633.shtml`. Vendored as `docs/etfspec.pdf`.
2. *Enigma Entry Pool: preliminary documentation*, Coda Music Technologies, 8 February 1996. Archived at `web.archive.org/web/19990203113959/http://www.codamusic.com:80/down/finale/eeppd.txt`. Vendored as `docs/eeppd.txt`.
3. Principally *denigma* (MIT), `github.com/chrisroode/denigma`, which credits Deguerre for the `score.dat` keystream; the *EnigmaXML documentation* (MIT), `github.com/Project-Attacca/enigmaxml-documentation`; and *musxdom* (MIT), `github.com/rpatters1/musxdom`. The full list, with what each contributed, is in `docs/REFERENCES.md`.
4. *Enigma Entry Pool: preliminary documentation* (see note 2), which records the term: “Certain situations like mirrors and voice 2 create complications.”
5. “Billion laughs attack”, Wikipedia: `en.wikipedia.org/wiki/Billion_laughs_attack`.
6. “XML external entity attack”, Wikipedia: `en.wikipedia.org/wiki/XML_external_entity_attack`.
7. Both expansions are Coda's, from the ETF specification (see note 1): “the entry duration in EDUs (Enigma Duration Units, 1024 == quarter note)” and “manual positioning in EVPUs (Enigma Virtual Page Units, 288 per inch)”.